Dense Subgraph Clustering and a New Cluster Ensemble Method

The-Anh Vu-Le[1], João Alfredo Cardoso Lamy[2], Tomás Alessi[2], Ian Chen[1], Minhyuk Park[1], Elfarouk Harb[1], George Chacko[1], and Tandy Warnow[1(✉)]

[1] Siebel School of Computing and Data Science, University of Illinois Urbana-Champaign, Urbana, IL 61801, USA
warnow@illinois.edu
[2] Insper Institute, Sao Paolo, Brazil
https://tandy.cs.illinois.edu

Abstract. We propose DSC-Flow-Iter, a new community detection algorithm that is based on iterative extraction of dense subgraphs. Although DSC-Flow-Iter leaves many nodes unclustered, it is competitive with leading methods and has high-precision and low-recall, making it complementary to modularity-based methods that typically have high recall but lower precision. Based on this observation, we introduce a novel cluster ensemble technique that combines DSC-Flow-Iter with modularity-based clustering, to provide improved accuracy. We show that our proposed pipeline, which uses this ensemble technique, outperforms its individual components and improves upon the baseline techniques on a large collection of synthetic networks.

Keywords: Dense subgraphs · Community detection · Cluster ensemble

1 Introduction

Community detection, also known as graph clustering, is the task of partitioning a network's vertices into groups that exhibit strong community structure. In general, the groups should exhibit strong edge density (i.e., many edges connecting nodes within groups), they should be well-connected (i.e., they should not have small edge cuts), and there should be relatively fewer edges between different groups [24]. Identifying these structures can be helpful for understanding the organization and function of complex systems across various domains, from bioinformatics, social networks, and scientific research networks [16].

Community detection is often posed as an optimization problem, with modularity one of the best known optimization problems [9]. Since these problems are generally NP-hard, finding optimal solutions is not reliable, and heuristics, such as Leiden [17] and Louvain [1], are used.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
H. Cherifi et al. (Eds.): COMPLEX NETWORKS 2025, SCI 1266, pp. 29–40, 2026.
https://doi.org/10.1007/978-3-032-16719-4_3

The Iterative k-Core (IKC) algorithm [23] is an example of an alternative approach, which focuses on the iterative extraction of high-quality subgraphs. Given the network, IKC finds a k-core (i.e., a subgraph in which all nodes have degree at least k) for the largest achieved value of k, treats the k-core as a community and then removes it, and recurses until a stopping rule applies (e.g., no k-cores for $k > 10$ are left). This iterative approach provides the motivation for our proposed method, which replaces the k-core concept with a different measure of density. We examine clustering methods that use greedy iterative approaches based on seeking subgraphs that have the largest edge density [5], where the density of a subgraph is the ratio of the number of edges to the number of vertices.

The Dense Subgraph Decomposition procedure described in [5] partitions the vertex set of the input graph $G = (V, E)$ into a series of subsets based on decreasing density. The process begins by identifying S_1, which is the unique maximal densest subgraph in the graph G. Subsequently, for each step i, the procedure identifies the maximal set S from the remaining vertices that maximizes the function $(|E(S)| + |E(S, U_{i-1})|)/|S|$, where U_{i-1} is the union of all previously identified subsets $S_1, \ldots, S_{i-1}$, $E(S)$ is the set of edges between vertices of S (i.e., $E(S) = \{\{u, v\} \in E \mid u, v \in S\}$), and $E(S, S')$ is the set of edges between vertices of S and S' (i.e., $E(S, S') = \{\{u, v\} \in E \mid u \in S, v \in S'\}$). Notice that $|E(S)|/|S|$ is the density of the subgraph induced by S. Hence, this approach differs from a simple iterative extraction of the densest subgraph since it also accounts for the edges connecting a candidate subgraph to the parts that have already been extracted.

Harb and colleagues proposed two main approaches to compute a dense subgraph decomposition: FISTA [5], which solves a convex relaxation of the problem using the Fast Iterative Shrinkage-Thresholding Algorithm (FISTA), and Flow [6], which solves a max-flow formulation of the problem using a push-relabel solver. The Flow technique is expected to provide better accuracy than FISTA.

We study the application of FISTA and Flow within an iterative framework, and find that our new approach (DSC-Flow-Iter), described in greater detail below, provides the best accuracy of these density-based methods. DSC-Flow-Iter is not quite as accurate as the best standard method we evaluated (i.e., Leiden-CPM, followed by postprocessing using WCC [11]), but a new cluster ensemble approach we present here that combines DSC-Flow-Iter with other techniques attains higher accuracy than all tested methods.

Due to space restrictions, full details and some results are provided in the Supplementary Materials, available at the DSC-Flow-Iter github site [19].

2 Materials and Methods

We present an overview of the methods we studied and the experiments we performed to evaluate these methods. The data used in this study are available from prior publications (see [21]). The software for the DSC-Flow-Iter and ensemble methods are available at [19] and [10]. The software for other methods as well as the specific commands used are detailed in Supplementary Materials Section 2.

2.1 Datasets

Real-world networks We use a collection of 78 real-world networks sourced from the Netzschleuder database [15] and from [13]. These networks span various domains, including social networks, collaboration networks, web graphs, and communication networks. Their sizes range from 906 to 14 million of nodes. A complete list of these networks is available in the Supplementary Materials Section 1. For evaluating clustering accuracy of different methods, we excluded the 4 largest networks (`livejournal`, `orkut`, `bitcoin`, and `CEN`) and generated synthetic networks based on the remaining 74; the largest of these networks has 1.4 million nodes. For evaluating computational performance, we used all 78 networks.

Synthetic networks We use EC-SBM [20] to generate synthetic networks based on 74 of the real-world networks we studied. EC-SBM takes as input the parameters obtained from a clustered network and generates a synthetic network that aims to reproduce the cluster and network statistics. For this study, we used 74 of the real-world networks we studied, with the input clustering obtained using Stochastic Block Models (SBMs) computed using graph-tool [14], and then post-processed using Well Connected Clusters (WCC, see below) to ensure that the clusters are well-connected. This input clustering approach was selected because [20] established that it provides the closest fit to the network and clustering statistics.

2.2 Clustering Methods

We compared density-based clustering methods to standard clustering methods, including Leiden optimizing modularity (Leiden-Mod), Leiden optimizing under the Constant Potts Model (Leiden-CPM), Infomap, and IKC. Of these, only the density-based methods are new, and so we describe these further.

Density-based techniques We adapt the Flow and FISTA algorithms [5,6] as follows. Each of these methods computes values for the vertices so that all vertices with the same values will be put into the same cluster. The values produced by FISTA are approximate and cannot be used naively. Therefore, we apply a simple rounding heuristic, grouping nodes whose values round to the same integer. Since these groups may contain two or more connected components, we divide them into connected components, and call this method DSC-FISTA(int). In contrast, the values produced by Flow can be used directly (i.e., no rounding is needed, although division into connected components is still appropriate), and we call this method DSC-Flow.

An alternative way to use the two algorithms is through the iterative extraction of the densest subgraph. That is, given an input network, we can compute its approximate densest subgraph and remove the subgraph as a community, then recurse on the remaining graph until all vertices are accounted for. For Flow, the approximate densest subgraph contains all the nodes with the highest vertex values. For FISTA, the fractional peeling technique proposed in [5] is used

to get the approximate densest subgraph. Fractional peeling iteratively removes the vertex with the lowest current vertex value while updating the loads of its neighbors, and returns the densest subgraph generated during this process. When removing a subgraph, isolated vertices are produced; each such vertex is left as a singleton cluster. We name these methods DSC-Flow-Iter and DSC-FISTA-Iter, respectively.

2.3 Post-processing Techniques

We utilized the post-processing techniques described in [12,22], which are Connected Components (CC) and Well-Connected Clusters (WCC). The CC method splits a cluster that is internally disconnected into multiple clusters, each containing one of its connected components; we always apply this to any method that produces internally disconnected components. WCC is designed to refine an existing clustering by iteratively partitioning each cluster by removing minimum edge cuts until all the clusters are well-connected. We use the threshold from [13], which is that the size of its minimum edge-cut is greater than $\log_{10}(n)$ where n is the number of nodes inside the cluster. WCC is typically benign or beneficial when used with SBMs for community detection [12,22]. Here, we study the effect of WCC as a post-processing technique on the clustering methods we study.

2.4 The New Cluster Ensemble Technique

Cluster ensembles (also known as consensus clustering methods) are designed to take in a collection of clustering methods and return a clustering that achieves better stability or accuracy. There is a large literature on this problem, with recent surveys in [4,8,25], which we very briefly summarize here. The Cluster Ensemble Selection problem addresses the question of, given the collection of clustering methods, which ones should be included in the ensemble? Once the subset of clusterings is selected, different cluster ensembles use a variety of techniques to compute the consensus clustering. A common technique is to compute the co-occurrence matrix, which indicates for each pair of nodes the fraction of the input clusterings in which the two nodes are in the same cluster. This co-clustering matrix is then used to compute the consensus clustering. Note that this type of approach inherently assumes that all the input clusterings place all (or nearly all) of the network nodes into clusters (equivalently, do not produce clusters of size 1).

In our study, we are interested in learning whether density-based clustering methods, which we find do not cluster a large fraction of the network, can be used within a cluster ensemble approach. However, when some of the input clustering methods have very low node coverage, the co-occurrence matrix may incorrectly identify pairs that are strongly supported as having low support. In this case, the ensemble methods are likely to have poor accuracy. For this reason, we designed a new cluster ensemble method that we now describe.

Given a network $N_0 = (V, E)$, an arbitrary set of input clusterings, and a threshold value $t \geq 0$, we compute an ensemble clustering as follows. First, we build a weighted network N_1, initialized to N_0. Some of the edges will be removed and the ones that remain will be weighted based on the following protocol. For an edge $e = \{u, v\}$ in N_0 (and hence in N_1), we set $\tilde{M}_e$ to be the number of input clusterings that assign both u and v to the same cluster and M_e is the number of input clusterings that assign u and v each to a non-singleton cluster. We define the weight w_e of an edge $e = \{v, e\} \in E$ as follows. If no input clustering clusters u and v together, then we set $w_e = 0$. For all other edges e, we set $w_e = \tilde{M}_e / M_e$; these edges all have positive weight. The edge e is deleted if $w_e < t$.

We then run a final community detection algorithm to cluster N_1. One approach is to cluster the weighted network N_1, which requires that the final clustering technique can handle weighted networks. Another approach is to cluster the unweighted version of N_1.

2.5 Evaluation

To evaluate the methods, we compare them against the ground-truth partitions of our synthetic networks. We use three metrics: Adjusted Mutual Information (AMI) [18], Adjusted Rand Index (ARI) [7], and Normalized Mutual Information (NMI) [2]. We also use False Positive Rate (FPR), precision, and recall, where we represent a clustering by the equivalence relation on pairs of nodes that it defines, and we score an estimated clustering using this representation.

2.6 Experiments

We conducted four experiments:

- Experiment 1: We compare different density-based techniques and establish that DSC-Flow-Iter is the best of all the density-based techniques we studied.
- Experiment 2: We compare DSC-Flow-Iter with previous community detection methods.
- Experiment 3: Cluster ensemble design and evaluation.
- Experiment 4: We benchmark the runtime of the proposed pipeline on real-world networks.

All experiments were conducted on the Illinois Campus Cluster, using nodes with 256GB of RAM. The computation time for each task (running a community detection algorithm on a network, or running the post-processing treatment on a clustering result) was limited to 3 days.

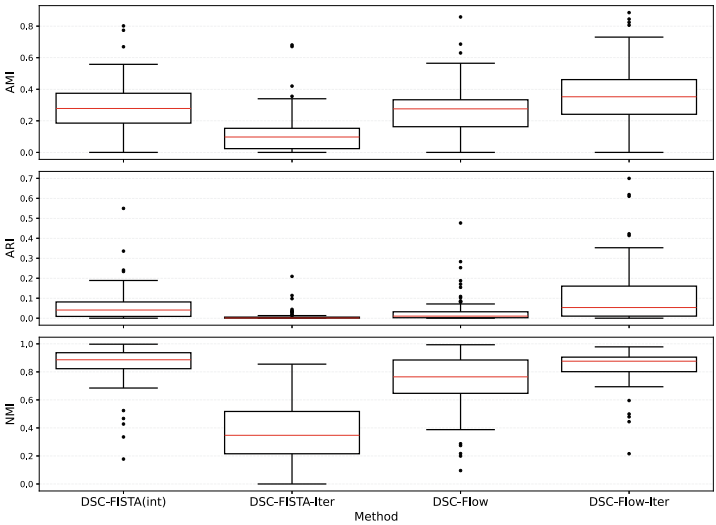


Fig. 1. Accuracy of density-based clustering methods on synthetic networks. The results are on 74 synthetic EC-SBM networks with SBM+WCC ground-truth clustering. DSC-Flow-Iter is the method with the highest accuracy for all metrics.

3 Experimental results

3.1 Experiment 1: Comparing density-based methods

We examine four different density-based methods: DSC-Flow, DSC-Flow-Iter, DSC-FISTA(int), and DSC-FISTA-Iter. As shown in Fig. 1, DSC-Flow-Iter consistently achieves the highest accuracy across all three metrics. This result establishes that the iterative removal of the densest subgraph, as identified by the more precise Flow algorithm, is the most promising approach. Therefore, we select DSC-Flow-Iter as the representative method for our density-based approach in all subsequent experiments. However, as noted in Supplementary Materials Table S1, DSC-Flow-Iter has low node coverage, on average only putting 63.5% of the nodes into non-singleton clusters.

The performance differences between the methods can be understood by examining the vertex values they produce, as illustrated for a sample network in Fig. 2. The output of the FISTA algorithm forms a staircase-like pattern composed of many distinct steps connected by intermediate transitional values. This explains why integer rounding, although naive, is already an effective heuristic: the granular steps identified by FISTA are already well-separated, so the rounding process still preserves the underlying community structure. In contrast, the Flow algorithm produces a much coarser partitioning. As seen in the figure, a single, large step in the Flow output often encompasses and merges multiple, smaller steps from the FISTA output. For example, the first major plateau for Flow (at a value just under 4) corresponds to a region where FISTA identifies several distinct, lower-level density steps (at values around 1, 2, and 3). This

shows that the Flow method effectively groups together many of the more granular communities suggested by FISTA into larger, more discrete clusters.

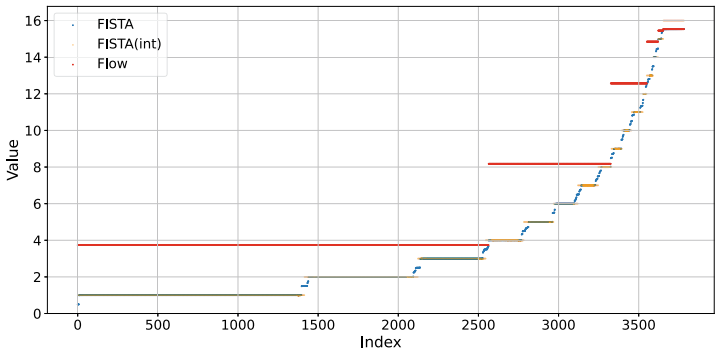


Fig. 2. Vertex values output by density-based techniques The result is on the EC-SBM synthetic network based on `bitcoin_alpha` and SBM+WCC ground-truth clustering.

3.2 Experiment 2: Evaluating DSC-Flow-Iter

We compared our best method, DSC-Flow-Iter, to Leiden-Mod, Leiden-CPM, Infomap, and IKC, all established community detection algorithms. The best parameter choices for IKC (the minimum value k that is considered) and Leiden-CPM (the resolution parameter value) were determined in a preliminary experiment (Supplementary Materials Figure S1).

Table 1 shows the accuracy of these methods without post-processing using WCC. Leiden-CPM(0.01) has the best AMI and ARI scores. Our proposed DSC-Flow-Iter is the next most accurate method for AMI and ARI accuracy, and the best for NMI. This demonstrates that DSC-Flow-Iter, although not the most accurate method, is competitive with other techniques.

We then investigated the accuracy of methods after post-processing by WCC. Since the synthetic network ground truth clusters are all well-connected, WCC is expected to be beneficial. As seen in Supplementary Materials Figure S2, the most accurate method is Leiden-CPM(0.01)+WCC, achieving the same top accuracy for AMI and NMI as Leiden-Mod+WCC, but achieving better ARI scores than Leiden-Mod+WCC. Based on this, we select Leiden-CPM(0.01)+WCC as our choice for use as the final clustering step in our ensemble approach.

3.3 Experiment 3: Designing the Cluster Ensemble Method

The cluster ensemble method we design takes as input a network, a set of two or more clusterings, a threshold t, and a technique for clustering the final edge-

Table 1. Accuracy of community detection methods on synthetic networks The results are on 74 synthetic EC-SBM networks with SBM+WCC ground-truth clustering. The best accuracy is boldfaced, the second-best accuracy is underlined.

	AMI			ARI			NMI		
	Med	Avg	Std	Med	Avg	Std	Med	Avg	Std
DSC-Flow-Iter	0.352	0.370	0.211	0.053	0.121	0.157	**0.876**	**0.833**	0.129
IKC(1)	0.297	0.307	0.167	0.019	0.060	0.104	0.816	0.802	0.132
Leiden-CPM(0.01)	**0.463**	**0.435**	0.273	**0.126**	**0.238**	0.250	0.854	0.830	0.135
Leiden-Mod	0.220	0.271	0.221	0.022	0.073	0.132	0.493	0.503	0.153
Infomap	0.167	0.233	0.245	0.010	0.081	0.163	0.490	0.454	0.303

weighted network. Based on Supplementary Materials Fig S2, we selected Leiden-CPM(0.01)+WCC as the clustering method for the final edge-weighted network. Here we describe the rest of the design.

Selecting the clustering methods to combine Since we are using Leiden-CPM for the final clustering method, we do not consider Leiden-CPM for the input clusterings. We sought clustering algorithms with complementary characteristics in terms of precision, recall, and FPR (see Supplementary Material Figure S3). DSC-Flow-Iter exhibits moderate precision and a low FPR, but also suffers from low recall. This indicates that while the pairs of nodes DSC-Flow-Iter places in the same community are very likely correct, it misses many other pairs that should be together. Conversely, Leiden-Mod shows a very high recall but has low precision and a high FPR, which suggests that modularity optimization tends to group nodes aggressively, correctly co-clustering many pairs but also incorrectly merging distinct communities, which is consistent with theoretical observations about the resolution limit made in [3]. IKC shows a profile similar to DSC-Flow-Iter but is outperformed on all metrics. Infomap is similar to Leiden-Mod but is also outperformed on all metrics, especially with respect to FPR. The complementary properties of DSC-Flow-Iter (high precision, low recall) and Leiden-Mod (low precision, high recall) make them good choices for an ensemble, and we use this combination in the subsequent experiment.

Comparing different cluster ensemble variants Based on the previous experiment, we explored the cluster ensemble approach using DSC-Flow-Iter and Leiden-Mod as the input clusterings and Leiden-CPM(0.01)+WCC to cluster the weighted network we produce. What remains to decide is the value of the threshold t and whether Leiden should consider the edge weights. We consider two values for t (0.5 and 1.0) and vary whether we consider edge weights, thus producing four variants of the cluster ensemble.

We summarize the trends we observed (see Supplementary Materials Figure S4 for full results). Setting $t = 0.5$ improved accuracy for all three criteria, but considering weights was slightly detrimental for accuracy. Thus, using the unweighted network generated from the combination with a threshold of $t = 0.5$

provides the best and most consistent gains. Furthermore, this variant improved accuracy in over 75% of the networks tested. Thus, we propose:

1. Stage 1: Run DSC-Flow-Iter on the input network
2. Stage 2: Run Leiden-Mod on the input network
3. Stage 3: Combine these clusterings to create an unweighted output network, retaining only the edges with weights at least threshold $t = 0.5$
4. Stage 4: Run Leiden-CPM(0.01) on this output network and apply WCC post-processing to the resulting clustering.

Table 2. Accuracy of the ensemble and all its constituents The results are on 74 synthetic EC-SBM networks with SBM+WCC ground-truth clustering. Leiden-CPM+WCC is run with resolution value 0.01. The ensemble method applies Leiden-CPM+WCC to the graph formed on Flow-Iter and Leiden-Mod, using threshold 0.5. The best accuracy is boldfaced.

	AMI			ARI			NMI		
	Med	Avg	Std	Med	Avg	Std	Med	Avg	Std
DSC-Flow-Iter	0.352	0.370	0.211	0.053	0.121	0.157	0.876	0.833	0.129
Leiden-Mod	0.220	0.271	0.221	0.022	0.073	0.132	0.493	0.503	0.153
Leiden-CPM+WCC	0.515	0.496	0.288	0.172	0.271	0.273	0.914	0.882	0.141
Ensemble	**0.572**	**0.516**	0.301	**0.225**	**0.308**	0.281	**0.921**	**0.894**	0.136

Results for the ensemble method As seen in Table 2, using our proposed ensemble method achieved better accuracy than its constituent methods, for all three criteria (AMI, ARI, and NMI). Specifically, running Leiden-CPM(0.01)+WCC on the modified network based on clustering of DSC-Flow-Iter and Leiden-Mod gave a more accurate clustering than running on the original network. Furthermore, as we have previously identified Leiden-CPM(0.01)+WCC as the most accurate single-algorithm method, this indicates that the selected ensemble method provides an improvement over all the single-algorithm techniques.

3.4 Experiment 4: Runtime for the Pipeline

We evaluated the runtime of our proposed pipeline on 78 real-world networks. The pipeline failed to complete on five large networks (`google_web`, `livejournal`, `orkut`, `bitcoin`, and `CEN`) ranging from ∼850 K to ∼14 M nodes, due to OOM (out-of-memory) errors during Stage 1.

As shown in Fig. 3, the runtime of Stage 1 exhibits a power-law relationship with the network size, appearing as a linear trend on the log-log scale, but the combined runtime of the remaining stages grows more slowly. This difference in growth rates means that while the rest of the pipeline (Stages 2, 3, and 4) can dominate the runtime for smaller networks, Stage 1 becomes the clear

bottleneck and the most time-consuming step for larger networks. This is also evident from the breakdown of the runtime in Fig. 4. These trends highlight that the scalability of our current pipeline is limited by the DSC-Flow-Iter stage.

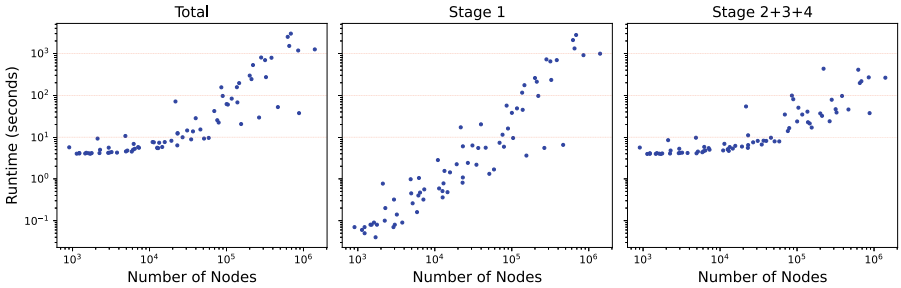


Fig. 3. Scatter plot of runtimes of the pipeline on real-world networks Both x-axis and y-axis are in log-scale. The results are on 73 real-world networks. Stage 1 runs DSC-Flow-Iter. We observe that the pipeline tends to run longer as network size grows. Furthermore, the dominant factor is Stage 1 (DSC-Flow-Iter) for larger networks but not for smaller networks.

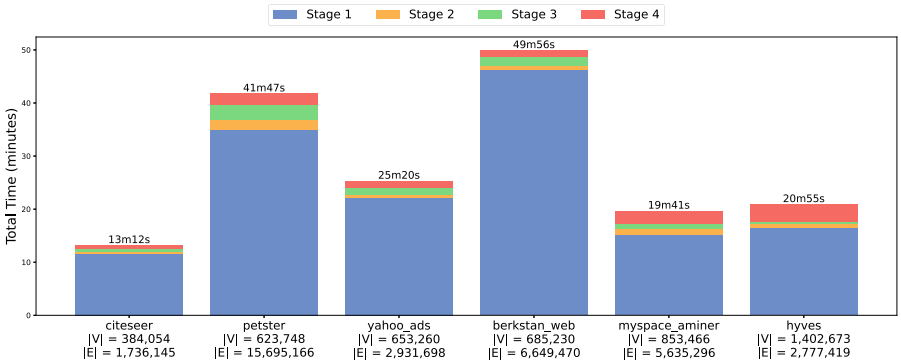


Fig. 4. Breakdown of pipeline runtime by stage on networks that used at least 800 seconds. Stage 1 runs DSC-Flow-Iter. Stage 2 runs Leiden-Mod. Stage 3 uses the clusterings from the previous two stages to generate the unweighted network. Stage 4 applies the final clustering (Leiden-CPM(0.01)+WCC)).

4 Conclusion

In this work, we have introduced a novel community detection algorithm, DSC-Flow-Iter, based on the iterative extraction of dense subgraphs. Our experiments showed that this method achieved high accuracy, second only to Leiden-CPM(0.01), on a suite of synthetic networks. More importantly, we identified the complementary nature of our density-based method (high precision)

and modularity-based methods like Leiden-Mod (high recall). This insight led us to propose a new cluster ensemble method that integrates these different approaches. The resulting pipeline, which combines the outputs of DSC-Flow-Iter and Leiden-Mod to generate a refined network, followed by a final clustering with Leiden-CPM(0.01) and post-processing with WCC, improved on the best single-algorithm approach. Overall, therefore, we find that while community detection via dense subgraphs is not the best on its own, it can contribute within an ensemble of diverse methods, which together achieve a better accuracy.

The ensemble technique can also be described as a way of using the input clusterings (here, Leiden-Mod and DSC-Flow-Iter) to identify the unreliable edges in the network, so that they can be removed before the network is clustered (here by Leiden-CPM(0.01)+WCC). The entire improvement in accuracy over Leiden-CPM(0.01)+WCC, therefore, is due to the identification and deletion of these unreliable edges. We also note that the specific methods used as the input clusterings *does matter*, as replacing Leiden-Mod by Infomap (Supplementary Materials Figure S5) did *not* produce as large an improvement in accuracy over Leiden-CPM(0.01)+WCC as using Leiden-mod.

Thus, future work should explore other ways of designing cluster ensemble methods, both for the selection of the input clusterings and in how they are used. Future work also needs to explore a larger set of synthetic networks to better understand the specific network properties where this and other cluster ensemble techniques provide benefits. Since the pipeline has a high memory requirement on some large networks, due to the DSC-Flow-Iter step, a new implementation that addresses memory usage is needed.

Funding This work was supported in part by the Illinois-Insper partnership and by NSF grant 2402559 to TW and GC.

References

1. Blondel, V.D., Guillaume, J.L., Lambiotte, R., Lefebvre, E.: Fast unfolding of communities in large networks. J. Stat. Mech.: Theory Exp. **2008**(10), P10,008 (2008)
2. Forbes, A.D.: Classification-algorithm evaluation: five performance measures based on confusion matrices. J. Clin. Monit. **11**(3), 189–206 (1995)
3. Fortunato, S., Barthélemy, M.: Resolution limit in community detection. Proc. Natl. Acad. Sci. **104**(1), 36–41 (2007)
4. Golalipour, K., Akbari, E., Hamidi, S.S., Lee, M., Enayatifar, R.: From clustering to clustering ensemble selection: a review. Eng. Appl. Artif. Intell. **104**, 104,388 (2021)
5. Harb, E., Quanrud, K., Chekuri, C.: Faster and scalable algorithms for densest subgraph and decomposition. Adv. Neural. Inf. Process. Syst. **35**, 26966–26979 (2022)
6. Harb, E., Yassin, Y., Chekuri, C.: Corporate needs you to find the difference: revisiting submodular and supermodular ratio optimization problems (2025). ArXiv:2505.17443
7. Hubert, L., Arabie, P.: Comparing partitions. J. Classif. **2**, 193–218 (1985)

8. Hussain, M.T., Halappanavar, M., Chatterjee, S., Radicchi, F., Fortunato, S., Azad, A.: Parallel median consensus clustering in complex networks. Sci. Rep. **15**(1), 3788 (2025)
9. Newman, M.E.: Detecting community structure in networks. Eur. Phys. J. B **38**, 321–330 (2004)
10. Park, M.: ClusterMerger (2025). https://github.com/illinois-or-research-analytics/ClusterMerger
11. Park, M., Feng, D.W., Digra, S., Vu-Le, T.A., Chacko, G., Warnow, T.: Improved community detection using stochastic block models (2024). arXiv:2408.10464
12. Park, M., Feng, D.W., Digra, S., Vu-Le, T.A., Chacko, G., Warnow, T.: Improved community detection using stochastic block models. In: International Conference on Complex Networks and Their Applications, pp. 103–114. Springer, Heidelberg, Germany (2024)
13. Park, M., Tabatabaee, Y., Ramavarapu, V., Liu, B., Pailodi, V.K., Ramachandran, R., Korobskiy, D., Ayres, F., Chacko, G., Warnow, T.: Well-connectedness and community detection. PLOS Complex Syst. **1**(3), 1–25 (2024)
14. Peixoto, T.P.: The graph-tool python library. Figshare (2014). http://figshare.com/articles/graph_tool/1164194 (2014)
15. Peixoto, T.P.: The Netzschleuder network catalogue and repository (2020). https://networks.skewed.de
16. Su, X., Xue, S., Liu, F., Wu, J., Yang, J., Zhou, C., Hu, W., Paris, C., Nepal, S., Jin, D., et al.: A comprehensive survey on community detection with deep learning. IEEE Trans. Neural Netw. Learn. Syst. **35**, 4682–4702 (2022)
17. Traag, V.A., Waltman, L., Van Eck, N.J.: From Louvain to Leiden: guaranteeing well-connected communities. Sci. Rep. **9**(1), 1–12 (2019)
18. Vinh, N.X., Epps, J., Bailey, J.: Information theoretic measures for clusterings comparison: is a correction for chance necessary? In: Proceedings of the 26th Annual International Conference on Machine Learning, pp. 1073–1080 (2009)
19. Vu-Le, T.A.: Dense subgraph clustering (2025). https://github.com/illinois-or-research-analytics/DSC
20. Vu-Le, T.A., Anne, L., Chacko, G., Warnow, T.: EC-SBM synthetic network generator. Appl. Netw. Sci. **10**(1), 15 (2025)
21. Vu-Le, T.A., Chacko, G., Warnow, T.: EC-SBM benchmark networks (2025). https://doi.org/10.13012/B2IDB-3284069_V1. https://doi.org/10.13012/B2IDB-3284069_V1
22. Vu-Le, T.A., Park, M., Chen, I., Chacko, G., Warnow, T.: Using stochastic block models for community detection: the issue of edge-connectivity (2025). arXiv:2508.03843
23. Wedell, E., Park, M., Korobskiy, D., Warnow, T., Chacko, G.: Center-periphery structure in research communities. Quant. Sci. Stud. **3**(1), 289–314 (2022)
24. Yang, J., Leskovec, J.: Defining and evaluating network communities based on ground-truth. In: Proceedings of the ACM SIGKDD Workshop on Mining Data Semantics, pp. 1–8 (2012)
25. Zhang, M.: Weighted clustering ensemble: a review. Pattern Recognit. **124**, 108,428 (2022)